Person 3: `session.py` (keys and the encrypted channel)

Your job in one line: once the drones are authenticated, turn the Diffie-Hellman secret into proper keys, then protect every data packet: secret (confidentiality), unchanged (integrity), and not replayed.

Run your demo: `python session.py`

---

1. What to say (about 3 minutes)

> "After the handshake, both drones share a Diffie-Hellman secret. We **don't** use it directly as a key, because it isn't uniformly random and we need two different keys. We pass it through **HKDF-SHA256**, a key-derivation function, with both nonces as salt. That gives a fresh 32-byte **encryption key** and a 32-byte **MAC key** every session.
>
> The MAC key makes the **key-confirmation tag** in message M3 (HMAC-SHA256 over everything both drones saw).
>
> The encryption key is used with **AES-256-GCM**, which is **authenticated encryption**: one operation gives confidentiality *and* integrity. Every packet has a **sequence number**, which is used inside the nonce so a nonce never repeats, and is checked so an old packet can't be replayed. If an attacker flips even one bit, the GCM tag fails and the packet is dropped."

Then run `python session.py`: it encrypts, decrypts, rejects a replay and rejects a flipped bit.

---

2. Libraries in this file

Import	Where it's from	Why we need it
`os`	stdlib	Only for the demo: a random test key
`hashes`	`cryptography`	`hashes.SHA256()`, the hash used in HKDF and HMAC
`hmac`	`cryptography`	HMAC-SHA256 for the key-confirmation tag
`HKDF`	`cryptography`	Key derivation: DH secret → two clean keys
`AESGCM`	`cryptography`	AES-256 in Galois/Counter Mode = authenticated encryption
`InvalidSignature`	`cryptography`	Raised when an HMAC tag doesn't match
`InvalidTag`	`cryptography`	Raised when AES-GCM decryption finds tampering
`from gcs import AuthError, ser`	our file (Person 1)	Our error type and canonical encoding

---

3. Line by line

`derive_keys`, DH secret → keys

```
salt = bytes.fromhex(m1["n"]) + bytes.fromhex(m2["n"])
info = f"{m1['from']}|{m1['to']}".encode()
okm = HKDF(algorithm=hashes.SHA256(), length=64, salt=salt, info=info).derive(dh_secret)
transcript = ser(m1) + ser(m2)
return okm[:32], okm[32:], transcript
```

- `salt` = both drones' nonces. Even if a DH secret somehow repeated, the keys would still be new.
- `info` = the two drone IDs. The keys are **bound** to this pair of drones (context separation).
- HKDF works in two steps. **Extract** turns the DH output into a uniformly random key, and **expand** stretches it to 64 bytes. We split those into `K_enc` (first 32) and `K_mac` (last 32).
- `transcript` is everything both sides saw (M1 and M2). The confirmation tag covers it, so the two drones confirm they saw *the same* messages.

`drone.py` calls this in `finish()` (A's side) and `confirm()` (B's side). Both get identical keys.

`make_tag` / `check_tag`, key confirmation (message M3)

```
h = hmac.HMAC(k_mac, hashes.SHA256())
h.update(b"A-confirm" + transcript)
return h.finalize().hex()
```

HMAC = a MAC built from a hash function. Only someone with `K_mac`, which means someone who did the DH, can make this tag. The label `"A-confirm"` says what the tag is for (domain separation).

```
h.verify(bytes.fromhex(tag_hex))     # constant-time compare
```

`verify` compares in **constant time**. A normal `==` stops at the first wrong byte, and an attacker can time that to guess the tag byte by byte. It raises `InvalidSignature` if wrong, or `ValueError` for broken hex, and we convert both to `False`.

`Session.__init__`

```
self.key, self.role, self.peer = key, role, peer
self.send_seq = 0      # my outgoing counter
self.recv_seq = -1     # highest counter accepted
```

`role` is 0 for the initiator (A) and 1 for the responder (B). It's used in the nonce, explained below.

`seal`, encrypt one packet

```
self.send_seq += 1
nonce = bytes([self.role]) + self.send_seq.to_bytes(11, "big")
aad = str(self.send_seq).encode()
return {"seq": self.send_seq, "ct": AESGCM(self.key).encrypt(nonce, plaintext, aad).hex()}
```

- AES-GCM needs a 12-byte nonce that **must never repeat under the same key**. Our nonce is 1 byte of direction (role) followed by an 11-byte counter. The counter only goes up, so the nonce never repeats.
- Why the role byte? Both directions use the same key. Without it, A's packet #1 and B's packet #1 would have the same nonce, which is a catastrophic nonce reuse.
- `aad` = "additional authenticated data". The sequence number isn't secret, but it **is** protected: change it and decryption fails.
- `encrypt` returns ciphertext plus a 16-byte authentication tag.

`open`, decrypt one packet

```
if seq <= self.recv_seq:
    raise AuthError("replayed/old message")
...
pt = AESGCM(self.key).decrypt(nonce, bytes.fromhex(msg["ct"]), str(seq).encode())
except (InvalidTag, ValueError):
    raise AuthError("integrity check failed (tampered ciphertext)")
self.recv_seq = seq
```

1. **Replay check:** the sequence number must be higher than anything accepted so far.
2. **Decrypt with the *peer's* direction byte** (`1 - self.role`), so the nonce matches what the peer used.
3. Any change to the ciphertext, tag, or sequence number raises `InvalidTag`, which becomes an `AuthError`.
4. `recv_seq` is updated **only after** the packet checks out, so a forged packet with a huge sequence number can't block real packets.

---

4. How this file links to the others

```
gcs.py ──(AuthError, ser)──► session.py ──(derive_keys, make_tag, check_tag, Session)──► drone.py
```

- `drone.py`'s `finish()` calls `derive_keys` and `make_tag`, then makes `Session(k_enc, 0, ...)`.
- `drone.py`'s `confirm()` calls `derive_keys` and `check_tag`, then makes `Session(k_enc, 1, ...)`.
- `attacks.py` uses `.seal()` and `.open()` to test replay and tampering.

---

5. Lecture-plan topics (CS428) and how to say them

Lecture topic	Where	How to explain it
Authenticated Encryption	`seal` / `open` (AES-GCM)	"One primitive gives confidentiality + integrity; a modified ciphertext is rejected, never decrypted."
CCA-Secure Encryption, **Chosen-Ciphertext Attacks**	`open` rejects tampered packets	"An attacker who submits modified ciphertexts learns nothing, because they're all rejected. AE implies CCA security."
Chosen-Plaintext Attacks and CPA-Security	unique nonce per packet	"Encrypting the same message twice gives different ciphertexts, because of different nonces. Deterministic encryption would leak 'same command sent again'."
Security for Multiple Encryptions	counter nonce	"Many messages under one key stay secure only if the nonce never repeats, so we guarantee that with a counter."
Modes of Operation	GCM	"GCM = CTR mode for encryption + GHASH for authentication. CTR turns the AES block cipher into a stream cipher."
Pseudorandom Generators, Stream Ciphers & Block Cipher	AES, CTR	"AES is a pseudorandom permutation; CTR mode makes a keystream from it."
CPA-Secure Encryption from Pseudorandom Functions	AES-CTR	"Ciphertext = message XOR F_k(nonce‖counter). That's exactly the PRF-based construction from class."
Fixed-Length MAC, Domain Extension	HMAC	"HMAC extends a fixed-length MAC to messages of any length."
Message Authentication Using Hash Functions, **Merkle–Damgård Transform**	HMAC-SHA256	"SHA-256 is a Merkle–Damgård hash. HMAC's nested construction avoids its length-extension weakness."
The Random-Oracle Model, **CDH-Based KEM in the Random-Oracle Model**	HKDF on the DH secret	"Hashing a DH secret into a key is the CDH-based KEM from class. Its security proof treats the hash as a random oracle."
Indistinguishability	AES-GCM ciphertexts	"An attacker can't tell which of two messages was encrypted."

6. Likely questions

- **Why not use the DH secret directly as the AES key?** It's a curve point encoding, not uniformly random, and we need two independent keys. HKDF solves both.
- **Why AES-GCM and not AES-CBC + HMAC?** GCM is one standard AEAD (used in TLS 1.3), fast, and has hardware support. CBC + MAC is easy to get wrong (padding-oracle attacks).
- **What if a nonce repeats?** XORing two ciphertexts would reveal XOR of the plaintexts, and the GHASH key leaks, so the attacker could forge packets. That's why the counter and the role byte exist.
- **What happens to packets that arrive out of order?** An older sequence number is dropped. That's simple and safe. A real radio link would use a **sliding window** (like IPsec/DTLS) to accept slightly reordered packets.

- **Why HMAC verify and not `==`?** Constant time. `==` leaks timing information that can be used to guess the tag.